# PATCHWORK

**WEB TECHNOLOGY PROJECT REPORT, V SEM. B.Tech**
**FLEXICORE-3131**

**SUBMITTED TO**
**SCHOOL OF COMPUTER ENGINEERING, M.I.T., MANIPAL**


**By**
**Pagavath D P (Reg. No. 240953676; Roll No. 45)**

**Man Mohit Kumar (Reg. No. 240953402; Roll No. 44)**

**Under the guidance of: Dr. Roshan David Jathanna**


**November 2026**

**GitHub: https://github.com/pagavath-dp/PatchWork**


MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
*(A constituent institution of MAHE, Manipal)*

# Team Details and Division of Work

| Sl. | Name | Reg. No. | Modules / Pages designed | Signature |
|---|---|---|---|---|
| 1 | Pagavath D P | 240953676 | 1. GitHub OAuth authentication and server-side sessions.<br>2. Workspace creation, membership management and invitations.<br>3. Role-based authorization and permission enforcement.<br>4. Task creation, editing, assignment and status management.<br>5. Kanban board and list functionality.<br>6. Task revision checks, conflict handling and task activity recording. | Pagavath D P |
| 2 | Man Mohit Kumar | 240953402 | 1. GitHub App integration and repository connection management.<br>2. Repository metadata and saved issue/pull-request snapshots.<br>3. GitHub issue and pull-request linking.<br>4. Repository activity and combined team timeline.<br>5. Explainable project-health analysis and findings.<br>6. GitHub activity processing and related integration logic. | Man Mohit Kumar |

# 1. Introduction and Project Overview

PatchWork is a web application for a small software team to coordinate tasks and review engineering activity in the same workspace. Users sign in with GitHub, create or join workspaces, manage tasks in a list or Kanban board, and view activity associated with a connected repository. The application also points out project risks such as overdue tasks and inactive pull requests. The GitHub OAuth App used for sign-in is separate from the GitHub App used for repository access [1, 5, 6].

The project is organized as an npm workspace with a Next.js client, an Express API, shared validation contracts, and a Prisma/PostgreSQL data package. Its source is available at https://github.com/pagavath-dp/PatchWork. The repository includes local setup instructions, but no verified production deployment is documented [3].

## 1.1 Intended Users

PatchWork is intended for student project teams and other small development groups. Workspace owners and administrators manage membership and repository connections; members can work on tasks; viewers have read access. The interface adapts to narrow screens and uses text labels alongside visual status cues. A formal accessibility evaluation has not yet been recorded.

## 1.2 Site Map

| Page | Main purpose |
|---|---|
| /login and /workspaces | GitHub sign-in and workspace selection or creation |
| /workspaces/[workspaceId] | Task list, Kanban board, task editor, and health panel |
| /workspaces/[workspaceId]/members | Membership, invitations, and role management |
| /workspaces/[workspaceId]/repository | Repository connection and saved GitHub data |
| /workspaces/[workspaceId]/activity | Combined task and GitHub team timeline |
| /invitations/[token] | Invitation review and acceptance by the intended account |

# 2. Problem Statement

Task boards show planned work, while issue and pull-request pages show repository work. When these views are separate, a team must manually reconcile task ownership, due dates, and code-review activity. It can also mistake an old GitHub snapshot for current project state. PatchWork addresses this by retaining tasks as first-class records, linking them to verified repository items, and showing the freshness and limits of the saved data. Its health findings identify specific records and suggest a next action instead of presenting an unexplained score.

# 3. Objectives

- Provide GitHub-based sign-in with a secure server-side session and a separate authorization flow for repository access.
- Allow users to create workspaces, invite teammates, and enforce owner, admin, member, and viewer permissions in the API.

- Persist tasks with priority, due date, assignee, and four statuses; support editing in list and Kanban views.
- Read repository metadata, issues, pull requests, and activity through manual GitHub App refresh.
- Record task changes and links, and merge them with saved GitHub activity in a paginated team timeline.
- Explain overdue work, stale pull requests, idle review-stage tasks, and uneven task assignment.

# 4. Requirements and Technologies Used

## 4.1 Functional Requirements

| Requirement | Implemented behaviour |
|---|---|
| Identity and access | GitHub OAuth login, server-side sessions, workspace invitations, and role checks |
| Task coordination | Create, list, edit, assign, prioritize, date, and move tasks across four statuses |
| Repository context | GitHub App connection, manual refresh, saved issues/PRs and activity, verified task links |
| Team overview | Combined activity timeline and evidence-based project-health findings |

## 4.2 Software and Development Environment

| Layer | Technology |
|---|---|
| Client | Next.js 16, React 19, TypeScript, Tailwind CSS 4 |
| API and contracts | Node.js 24.13, Express 5, Zod 4 shared schemas |
| Persistence | PostgreSQL 17, Prisma 7, @prisma/adapter-pg |
| External service | GitHub OAuth App for identity; GitHub App and REST API for repository data |
| Local setup | npm workspaces and Docker Compose; editor/IDE: VS Code |
| Testing | Vitest unit, contract, UI, and API integration test files |

The API uses shared Zod schemas for request validation. State-changing requests also pass same-origin checks. The local development and test databases are configured separately; the database-writing test suites require deliberate use of the test database [3, 4].

# 5. System Architecture and Database Design

## 5.1 Application Architecture

The browser uses a single origin. Next.js serves the pages and rewrites /auth/* and /api/* requests to the Express server. Express owns authentication, authorization, GitHub communication, and database access. The data package generates the Prisma client and uses the PostgreSQL driver adapter at runtime [3].

| Browser and web application | API and shared contracts | Data and external services |
|---|---|---|
| Next.js pages and React forms<br>Task list and Kanban<br>Members, repository, timeline, health | Express routes<br>Session and origin checks<br>Zod validation<br>Workspace services | Prisma → PostgreSQL<br>GitHub OAuth App → identity<br>GitHub App → repository data |

Figure 1. PatchWork request and data flow.

## 5.2 Database Design

The initial data model holds users, sessions, workspaces, memberships, and tasks. Later additive migrations introduce invitations, one active repository connection per workspace, repository snapshots, task links, and task activity. The main relationships are summarized below [3].

| Model | Purpose and main relationship |
|---|---|
| User, Session | A user may have multiple opaque sessions; only token hashes are stored. |
| Workspace, WorkspaceMember | A workspace has members with one role per user; it also stores the workspace time zone. |
| WorkspaceInvitation | A hashed invitation token targets an immutable GitHub account ID and records status and expiry. |
| Task, TaskActivity | Each task belongs to a workspace; immutable activity rows record actor, event kind, details, and time. |
| GitHubConnection, GitHubItem, GitHubActivity | A workspace has at most one connection; saved items/events carry a sync generation. |
| TaskGitHubLink | A task can be linked to a verified saved issue or pull request; unlinking retains the row. |

Active repository reads use the current connection and sync generation. A successful refresh advances that generation with its snapshot writes in one transaction. Rows from an earlier generation remain stored but are excluded from the active repository view. Historical task link/unlink events remain in TaskActivity [3].

# 6. Authentication, Workspace Management and Task Management

Primary responsibility: Pagavath D P.

## 6.1 Authentication and Sessions

The sign-in route starts the GitHub OAuth flow with random state, PKCE S256, and short-lived transaction cookies. After the callback is checked, the API retrieves the GitHub identity and creates an opaque session. PostgreSQL stores a SHA-256 hash of the session token; the browser receives the token in a host-only, HttpOnly, SameSite=Lax cookie. The cookie is Secure when the application runs over HTTPS. Logging out revokes the session [3, 5].

## 6.2 Workspaces, Membership and Invitations

Workspace creation inserts the workspace and its owner membership in one transaction. An owner or admin can invite a GitHub username after the API resolves it to an immutable GitHub account ID. The invitation token is shown once to the inviter and stored as a hash. The intended user must sign in with that same GitHub account before accepting. Invitations expire after seven days; accepted, rejected, or revoked invitations cannot be reused [3].

The server checks permissions again inside membership transactions. Owners can manage all roles. Admins can invite and manage members or viewers but cannot promote themselves or change other admins or owners. The last owner cannot be demoted or removed. A member assigned to tasks must be reassigned before removal. Workspace data is hidden from nonmembers, and viewers cannot perform task writes [3].

## 6.3 Tasks List and Kanban

Tasks contain a title, optional description, status, priority, due date, optional assignee, creator, and timestamps. The four statuses are TODO, IN_PROGRESS, IN_REVIEW, and DONE. The list and board use the same persisted task data; moving a card sends a validated status update to the API. The API checks that an assignee belongs to the same workspace before saving. Task reads are paginated and scoped to the workspace [3].

Edits carry an expected revision calculated from persisted task fields. If another edit changed the task first, the API returns HTTP 409 with TASK_CHANGED and the user must reload before saving. A no-op edit creates no activity event. Meaningful task creation and edits write TaskActivity in the same transaction as the task mutation. This avoids a saved task change without its corresponding audit event [3].

# 7. GitHub Integration, Activity Timeline and Project Health

Primary responsibility: Man Mohit Kumar.

## 7.1 GitHub App Connection and Snapshots

Repository access uses a separate GitHub App. An owner or admin selects a repository and completes the app authorization flow. The server checks the account and repository access, discards the short-lived user access token, and later uses an installation token for manual refresh. The browser does not receive these tokens. The app requests read access to repository metadata, issues, and pull requests [3, 6].

A refresh retrieves repository metadata, issues, pull requests, and issue/PR events with bounded pagination. The service validates the repository identity and complete response before publishing a new snapshot generation. Failed fetches or database writes leave the previous active generation available. Disconnect and reconnection retain historical rows; active reads and task links are filtered to the current connection and generation [3].

## 7.2 Task Links and Team Timeline

A task can be linked only to a verified GitHub issue or pull request in the active repository snapshot. Linking and unlinking write TaskActivity in the same transaction as the link change. The Team timeline merges these task events with current-generation GitHubActivity events. It checks workspace membership, sorts by time with stable tie-breaking, and serves bounded pages through an authenticated cursor. Task history remains visible when GitHub data is missing or unavailable; the response reports saved snapshot freshness separately [3].

## 7.3 Explainable Project Health

The health engine evaluates current task and saved pull-request data without generating a vague overall score. It flags unfinished tasks past their due date in the workspace time zone, review-stage tasks with no task update for at least three days, open pull requests inactive for at least seven days at the saved snapshot, and a substantial imbalance in assigned unfinished tasks. Findings include severity, explanation, evidence, and a suggested action. The health response marks missing GitHub data as unavailable and snapshots older than 24 hours as stale [3].

The rules do not know actual reviewer decisions, task effort, or team capacity. A saved pull request may have changed after the most recent refresh, so the finding states the snapshot time rather than claiming live GitHub status.

# 8. Implementation Details

## 8.1 Validation, Authorization and Transactions

Shared schemas in packages/contracts validate task, invitation, and repository requests. API routes authenticate the user, then services check workspace membership and role. The same check is made inside transactions that change tasks or membership. Workspace locks coordinate assignment with membership changes; task link and unlink transactions also lock the connection row before reading the active generation [3].

**Excerpt 1. Task revision check from apps/api/src/services.ts**

```
if (taskRevision(before) !== input.expectedRevision) {
  throw new HttpError(409, "TASK_CHANGED",
    "This task changed since you loaded it. Reload tasks before saving.");
}
const changes = changedFields(before, next);
if (changes.length === 0) return before;
```

The revision is a digest of the stored title, description, status, priority, due date, and assignee. It detects a stale edit without adding a revision column. Activity is written only after a persisted change, and within the same transaction [3].

## 8.2 Snapshot and Activity Handling

**Excerpt 2. Repository validation from apps/api/src/github-services.ts**

```
const snapshot = await gateway.fetchSnapshot(
  connection.installationId, connection.repositoryId, connection.fullName);
if (String(snapshot.repository.id) !== connection.repositoryId)
  throw new HttpError(403, "GITHUB_REPOSITORY_MISMATCH",
    "GitHub returned a different repository");
```

Snapshot publication is guarded by the connection identity and sync generation. The combined timeline carries the first page time bound and last item position in a signed cursor. If a refresh or reconnection changes the generation between pages, the API asks the client to reload rather than mixing event generations [3].

## 8.3 Main API Paths

| Path | Purpose |
|---|---|
| POST /api/v1/workspaces | Create workspace and owner membership |
| POST /api/v1/workspaces/:id/invitations | Create an invitation for a verified GitHub account |
| GET, POST /api/v1/workspaces/:id/tasks | List or create workspace tasks |
| PATCH /api/v1/workspaces/:id/tasks/:taskId | Edit a task with expected revision |
| GET /api/v1/workspaces/:id/activity | Read the combined team timeline |
| GET /api/v1/workspaces/:id/health | Read deterministic health findings |
| POST /api/v1/workspaces/:id/repository/refresh | Manually refresh the current repository |

# 9. Results and Screenshots

The application contains the pages and workflows described above. The team has reported local sign-in, invitation acceptance with a second account, workspace access, task editing, role restrictions, and persistence after refresh

.

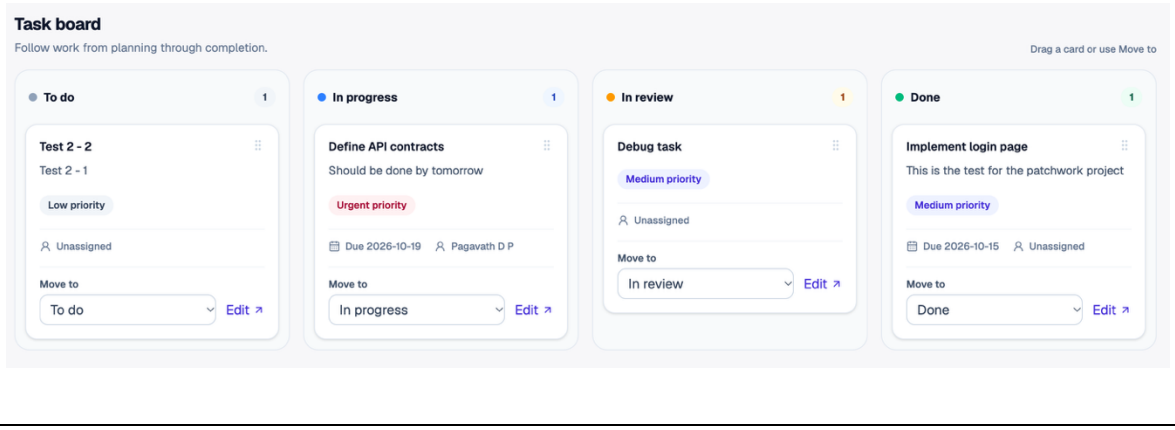



*Figure 2. Workspace dashboard and Kanban board.*

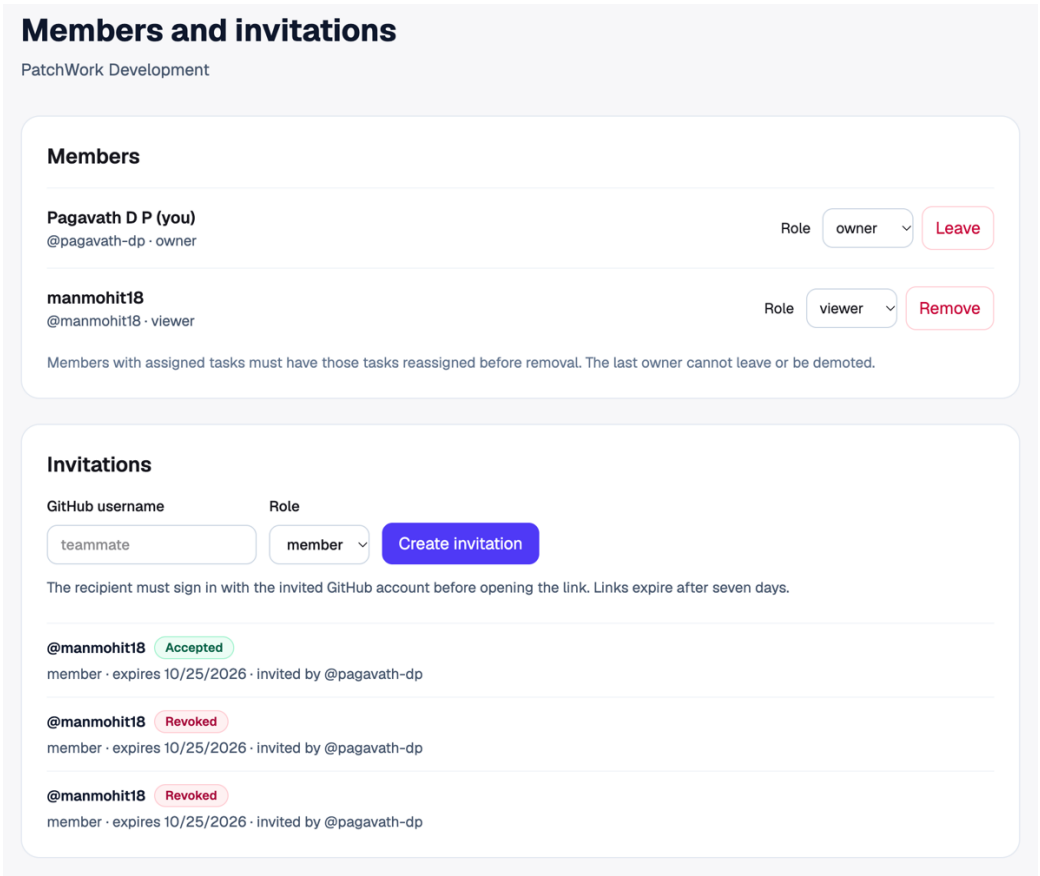



*Figure 3. Members and invitations.*

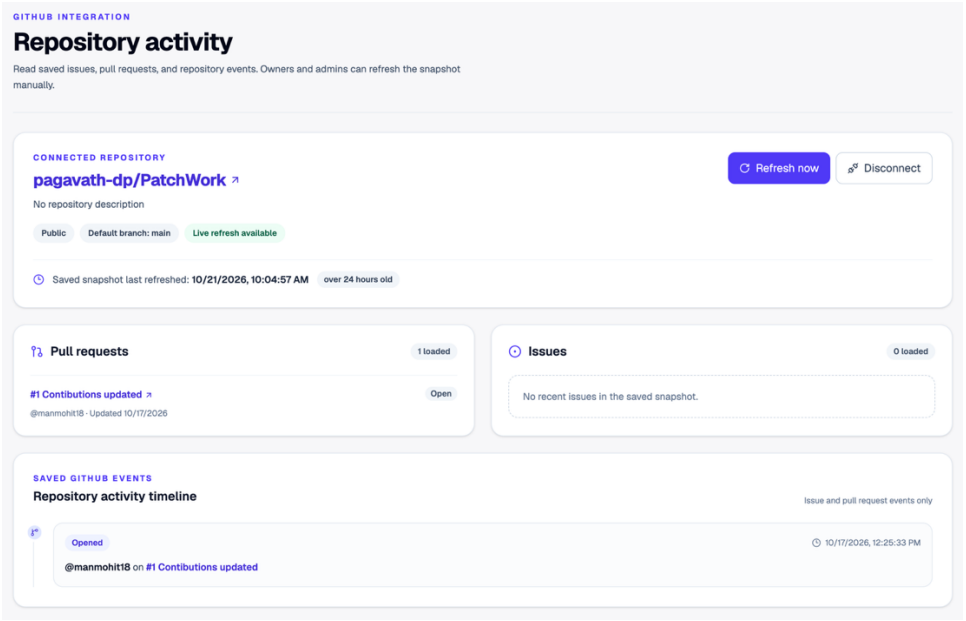



*Figure 4. Repository activity.*

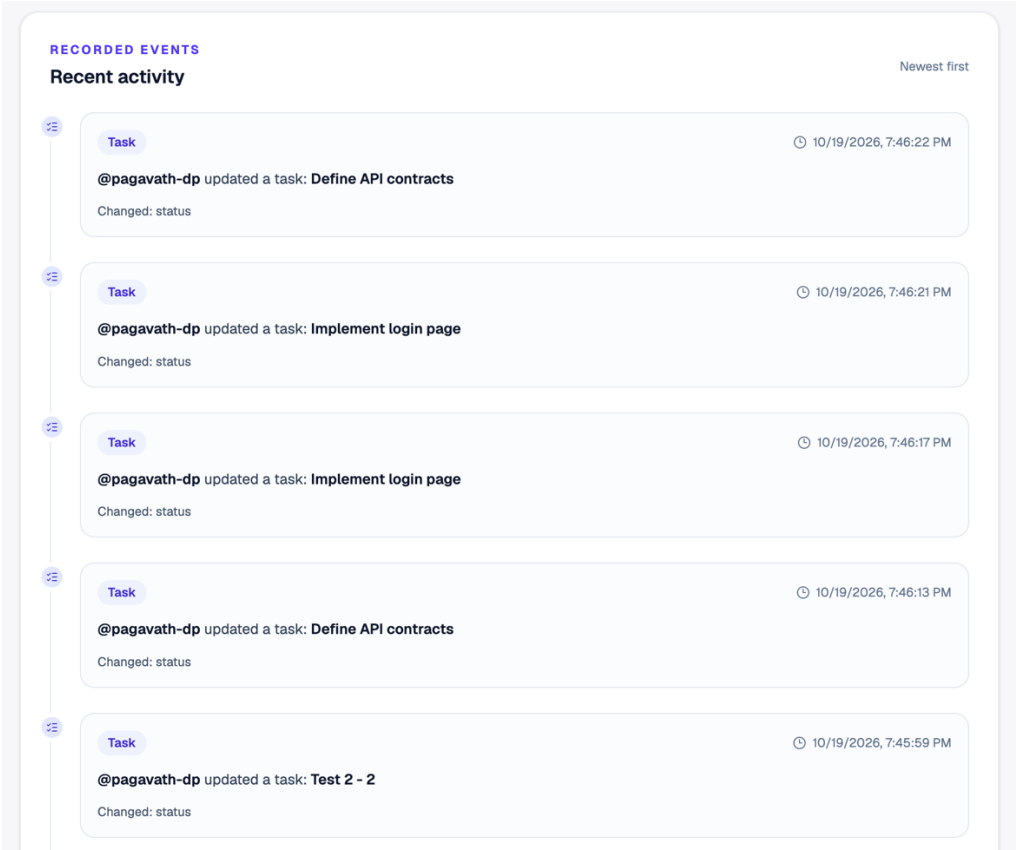



*Figure 5. Combined team timeline.*

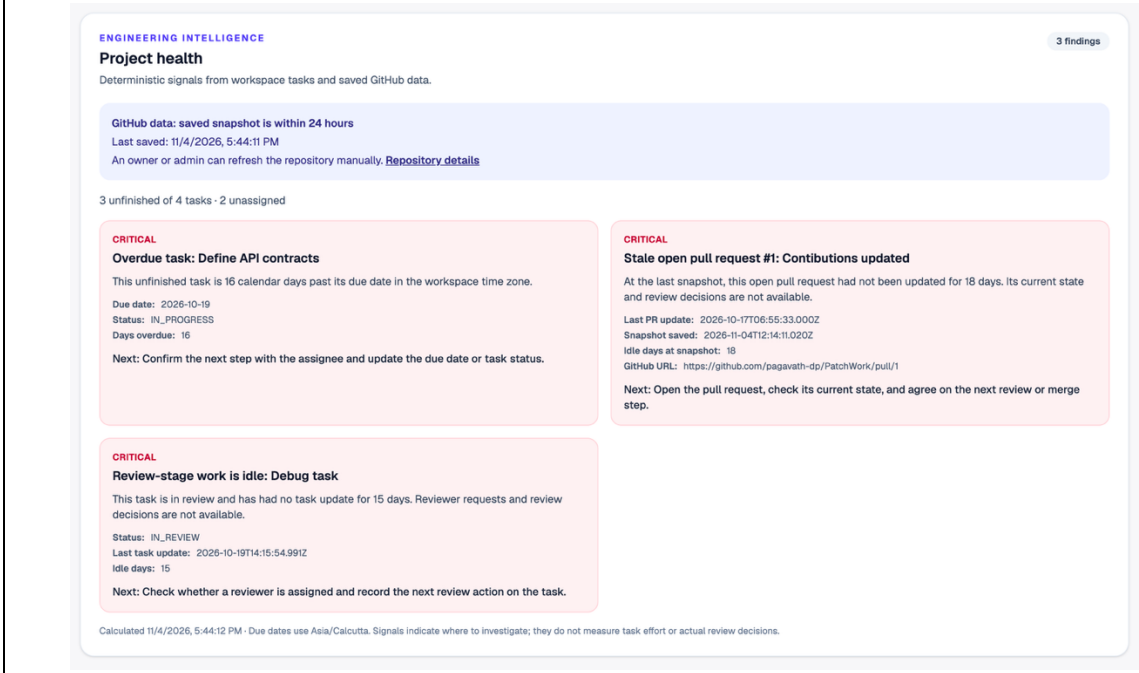



*Figure 6. Project health panel.*

# 10. Testing and Verification

The repository contains database-independent API unit tests, web component tests, contract tests, and a separate database-writing API integration suite. Relevant test files include apps/api/tests/workspace-membership.test.ts, activity.test.ts, health.test.ts, and web timeline tests. They cover invitation authorization, task event rollback and no-op behaviour, cursor ordering, missing GitHub data, and health rule boundaries. Test presence establishes coverage in source; the final submission should attach fresh command output for the exact commit being submitted [3, 7].

| Verification area | Current evidence and limit |
|---|---|
| Local two-account workflow | Team-reported browser check of invitation acceptance, shared workspace access, task editing, role permissions and refresh persistence. |
| API and UI tests | Unit, contract and web suites exist; do not treat report creation as a fresh test run. |
| GitHub App live workflow | Connection, refresh and failure recovery need final verification with a configured installation. |
| PostgreSQL concurrency | A successful database-backed race verification has not been established. |
| Accessibility and deployment | Formal assistive-technology audit and hosted HTTPS deployment are not documented. |

Before submission, run the approved checks against the final source and record their results. Database-writing suites must target the dedicated test database. The report does not claim that unverified integration or concurrency behaviour has passed.

# 11. Conclusion and Future Scope

PatchWork brings task coordination, GitHub repository context, team activity and project-health findings into one workspace. The team implemented these features with separate login and repository authorization flows, server-side permissions, persistent data and shared request validation. The most useful final checks are a live GitHub App walkthrough, fresh automated test results, and the screenshots listed in Section 9. Future work could add task archive or deletion, a commit feed, scheduled synchronization or webhooks, notifications, and a hosted deployment. These items are outside the present implementation.

# 12. References

[1] Next.js, App Router documentation. https://nextjs.org/docs/app

[2] Express, Routing guide. https://expressjs.com/en/guide/routing/

[3] PatchWork source code and project documentation. https://github.com/pagavath-dp/PatchWork; docs/architecture.md; docs/api-contracts.md.

[4] Prisma ORM 7, Migrate deploy reference. https://www.prisma.io/docs/cli/v7/migrate/deploy

[5] GitHub Docs, Authorizing OAuth apps. https://docs.github.com/en/apps/oauth-apps/building-oauth-apps/authorizing-oauth-apps

[6] GitHub Docs, Authenticating to the REST API. https://docs.github.com/en/rest/authentication

[7] Vitest, Getting Started. https://vitest.dev/guide/

[8] Tailwind CSS, Installation. https://tailwindcss.com/docs/installation/